# 实时状态缓存与并发通知

DebugStateStore 以设备、来源和字段三层键保存最新值，负责进程内实时缓存。它不负责来源定义、设备发现、历史记录和持久化。

适用分支 p/caijiamin/feat_debug_2th
基线提交 9010928ca51c6e766f75414c49c748338e0b7ecb
资料日期 2026 年 11 月 3 日

## 存储结构与更新

内部结构为 deviceId 到 sourceId 再到 fieldId 的可变 map。updateFields 对传入字段批量覆盖，null 表示删除该字段；未传入的其他字段保留原值。

空更新 map 直接返回。字段删除后，空的来源 map 与设备 map 会被清理，防止空容器持续堆积。全量替换来源数据时，不能只省略字段并期待旧值自动消失。

## 读取与对象边界

getFieldValue 返回单字段值；getFields 返回该来源 map 的副本。副本保护容器本身，不代表 Any 类型的嵌套对象已经深拷贝；来源宜提交不可变或独立复制的数据。

字段定义和实时值属于不同组件。读取前可以结合 Registry 验证来源绑定，不要仅凭 StateStore 中曾存在一个值就认定设备仍然在线。

## 同步与通知

更新和读取通过 synchronized 保护内部结构，写入后通知监听。监听者需要考虑回调线程与更新线程一致，且多次更新可能导致多次通知。

将 UI 刷新放到主线程并做合并，可以避免高频数据更新直接重绘。当前浮窗用 refreshPending 与 registryRefreshPending 合并任务，属于消费端的行为。

## 清理语义

clearDevice 删除设备所有来源值；clearDeviceSource 删除一台设备一个来源；clearSource 删除所有设备的同一来源；clear 删除全部实时缓存。

Receiver 调用清理时还会维护 Registry，单独调用 StateStore 不会自动删除设备定义。业务应通过正确所有者执行组合操作。

## 新鲜度与历史能力

StateStore 的值没有通用时间戳、TTL 或 stale 状态，因此“有值”不等于“刚更新”。相机旧链路的 receivedAtMillis 不能自动提供给所有通用字段。

长时间性能导出需要另行采样存储、保留策略和查询接口。将内存最新值 map 导出成 CSV 不能称为时间序列历史导出。

## 源码依据与版本回查

以下路径相对于 OBSBOT_Live 仓库。Impl 表示 business/debug/debug_impl/src/main/java/com/obsbot/live/business/debug，API 表示对应 debug_api 源码根。摘要为文件 SHA256 前 16 位。

Impl/data/state/DebugStateStore.kt
摘要 68fbce64e3f9762a；符号 DebugStateStore、updateFields、addListener、removeListener、notifyChanged、getFieldValue

Impl/data/receiver/DebugDataReceiverImpl.kt
摘要 a6d4b4eed780d74e；符号 DebugDataReceiverImpl、registerDevice、registerSource、registerFields、bindSource、unregisterDevice

Impl/ui/floating/FloatingWindowController.kt
摘要 e8c0d9931e28f09c；符号 FloatingWindowController、setCurrentCameraDeviceId、register、release、checkMainThread、addEnabledStateListener